

UNIWERSYTET GDAŃSKI WYDZIAŁ MATEMATYKI, FIZYKI I INFORMATYKI

Michał Ryduchowski

Numer albumu: 296112

Maciej Wojciechowski

Numer albumu: 292571

Maciej Nasiadka

Numer albumu: 292574

Kierunek studiów: Informatyka (profil praktyczny)

DIGITAL DUNGEONS - PLATFORMA DO TWORZENIA TEKSTOWYCH GIER RPG

Praca licencjacka

wykonana pod kierunkiem

dr inż. Łukasza Kusznera

Gdańsk 2026

Spis Treści

1. Wprowadzenie	4
1.1. Cel i zakres pracy	4
1.2. Motywacja powstania systemu	4
1.3. Definicja i specyfika gatunku text-based RPG	6
1.4. Struktura pracy	6
2. Analiza problemu i istniejących rozwiązań	6
2.1. Architektura i logika systemowa gier tekstowych	7
2.2. Narzędzia do tworzenia gier RPG	7
2.3. Platformy, które zajmują się dystrybucją gier stworzonych przez użytkowników	8
2.4. Wnioski z analizy istniejących rozwiązań	9
2.5. Założenia projektowe systemu	10
3. Analiza systemu	10
3.1. Wymagania funkcjonalne	11
3.2. Wymagania niefunkcjonalne	11
3.3. Aktorzy systemu	12
3.4. Diagram przypadków użycia	13
3.5. Opis scenariuszy przypadków użycia	14
3.6. Diagramy sekwencji	18
3.7. Model danych systemu	19
4. Projekt systemu	22
4.1. Architektura systemu	22
4.2. Projekt bazy danych	23
4.3. Projekt interfejsu użytkownika	24
4.4. Projekt modułów systemu	27
4.4.1. Moduł użytkowników	27
4.4.2. Moduł marketplace gier	27
4.4.3. Moduł edytora gier	28
4.4.4. Silnik rozgrywki	28
4.5. Projekt API systemu	28
5. Implementacja systemu	29
5.1. Wykorzystane technologie	29
5.1.1. Frontend	29
5.1.2. Backend	29

5.1.3. Baza danych.....	29
5.2. Implementacja interfejsu użytkownika.....	29
5.3. Implementacja edytora gier.....	30
5.4. Implementacja silnika rozgrywki	30
5.5. Implementacja marketplace.....	30
5.6. Implementacja systemu użytkowników	30
6. Testowanie systemu.....	31
6.1. Strategia testowania.....	31
6.2. Testy frontendowe	31
6.3. Testy backendowe.....	31
6.4. Testy funkcjonalne systemu	32
6.5. Wyniki testów.....	32
7. Podsumowanie i kierunki dalszego rozwoju.....	32
7.1. Podsumowanie realizacji projektu	32
7.2. Ograniczenia systemu	33
7.3. Możliwości dalszego rozwoju.....	33
8. Bibliografia.....	35

1. Wprowadzenie

Gry komputerowe są dziś nieodłącznym elementem życia społecznego. Według *Video Games Europe 2023 – Key Facts Report* [1] w 2023 roku, 53% populacji badanych krajów europejskich w wieku 6-64, regularnie sięgało po gry wideo w czasie wolnym. Według tego samego raportu, każdego tygodnia, objęci badaniem gracze grają w gry średnio 8,9 godziny.

Wraz ze wzrostem popularności gier komputerowych rośnie również ich liczba. Na platformę dystrybucji gier *Steam* w 2025 roku wpłynęło 19 993 gier [2]. Wskazuje to na dużą aktywność twórców.

Pomimo szerokiej dostępności silników i technologii, proces tworzenia i publikacji gry komputerowej nadal wymaga znajomości narzędzi programistycznych lub specjalistycznego oprogramowania. Stanowi to barierę wejścia dla osób bez doświadczenia technicznego, które chciałyby tworzyć własne projekty.

Jednym z podejść ograniczających złożoność procesu tworzenia gier jest wykorzystanie formy tekstowej. Gry typu text-based RPG opierają się głównie na narracji oraz interakcji tekstowej, co pozwala na ograniczenie wymagań technologicznych i skupienie się na warstwie fabularnej.

W odpowiedzi na zidentyfikowane ograniczenia opracowano system *Digital Dungeons* – platformę do tworzenia i publikacji tekstowych gier RPG. Ten serwis umożliwia tworzenie, testowanie oraz publikowanie gier komputerowych bez konieczności pisania kodu. Edytor graficzny i interfejs użytkownika no-code sprawiają, że osoby niezwiązane z technologią mogą stworzyć cyfrowy „świat” i opublikować go w sieci.

1.1. Cel i zakres pracy

Celem pracy jest dostarczenie osobom bez doświadczenia programistycznego prostego, intuicyjnego i darmowego systemu tworzenia i publikowania tekstowych gier typu Role-Playing Game (RPG). W ramach pracy opracowano również silnik uruchamiania wytworzonych w tym systemie gier z funkcją zapisu postępu rozgrywki.

Efektem końcowym jest serwis internetowy z edytorem graficznym gier, silnikiem rozgrywki oraz publicznym „sklepem” internetowym, który może być wypełniany grami stworzonymi przez społeczność, umożliwiającym skomentowanie, polubienie lub ewentualną edycję opublikowanej gry.

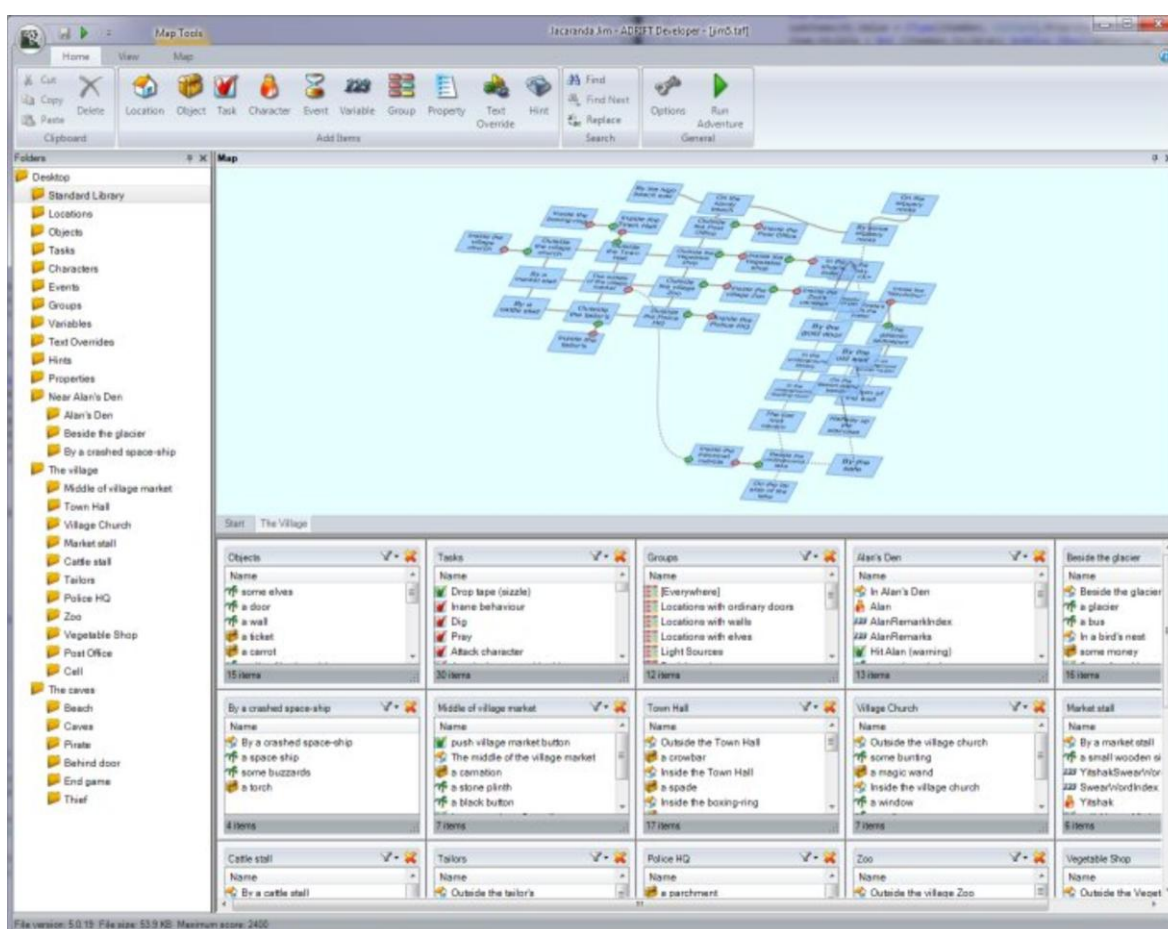
1.2. Motywacja powstania systemu

Dość popularnym sposobem integracji wśród młodzieży i młodych dorosłych są spotkania w celu przeprowadzania rozgrywek RPG. Gracze wcielają się w role postaci funkcjonujących w fikcyjnym świecie (np. czarodziejów, elfów), a przebieg rozgrywki kształtowany jest przez podejmowane przez nich decyzje oraz interakcje. Ogólnodostępne rozwiązania umożliwiające przeniesienie tego typu światów do środowiska cyfrowego są ograniczone.

Popularne silniki, takie jak *Unity*, czy *Unreal Engine*, nastawione są głównie na tworzenie efektownych światów 3D, animacji lub rzadziej prostszych gier 2D. Nawet mniej zaawansowany silnik *RPG Maker* zakłada, że świat będzie opierał się na siatce tekstur.

Rozwiązaniem umożliwiającym tworzenie tekstowych gier RPG jest *Text Adventure Development System* [3]. Ten silnik, jak i inne tego rodzaju, są narzędziami dla programistów lub bibliotekami kodu, wymagającymi solidnej znajomości co najmniej jednego języka programowania.

System *ADRIFT* [4] jest programem, który oferuje funkcjonalności zbliżone do opracowanego rozwiązania, jednak jego interfejs użytkownika charakteryzuje się dużą liczbą dostępnych opcji oraz złożoną strukturą, co może stanowić barierę dla użytkowników bez doświadczenia technicznego. Dodatkowo, wymaga pełnej instalacji (przez co może nie być kompatybilny z nowszymi systemami).



Rys. 1. Interfejs programu ADRIFT

W odpowiedzi na zidentyfikowane problemy zaprojektowano system tworzenia tekstowych gier RPG niewymagający znajomości programowania. W przeciwieństwie do wszystkich wyżej wymienionych rozwiązań, *Digital Dungeons* oferuje również własny internetowy „sklep” z grami stworzonymi przez społeczność użytkowników systemu.

1.3. Definicja i specyfika gatunku text-based RPG

Gry typu text-based RPG (tekstowe gry fabularne) są specyficzną kategorią gier komputerowych, w których głównym środkiem przekazu informacji jest tekst. W przeciwieństwie do gier opartych na grafice 2D lub 3D, interakcja z użytkownikiem odbywa się poprzez opisy słowne, komunikaty systemowe oraz decyzje podejmowane przez gracza.

Centralnym elementem tego typu gier jest rozbudowana warstwa narracyjna, definiująca otoczenie, postacie niezależne (NPC) oraz ciąg przyczynowo-skutkowy zdarzeń. Gracz wciela się w określoną rolę, stając się aktywnym uczestnikiem i współtwórcą opowieści, a jego wybory bezpośrednio kształtują dalszy przebieg rozgrywki.

Wyróżnia się dwa główne podejścia do interakcji w grach tekstowych. Pierwszym z nich są systemy oparte na parserze poleceń, w których użytkownik wprowadza komendy w języku naturalnym lub uproszczonej składni (np. „idź na północ”, „podnieś przedmiot”). Drugim podejściem są systemy wyboru (ang. choice-based), w których użytkownik podejmuje decyzje poprzez wybór jednej z predefiniowanych opcji przedstawionych przez system.

Ze względu na brak warstwy graficznej, gry tekstowe opierają się w dużym stopniu na wyobraźni użytkownika. Pozwala to na tworzenie złożonych światów i scenariuszy przy relatywnie niewielkich wymaganiach technicznych.

1.4. Struktura pracy

Praca składa się z ośmiu rozdziałów:

Rozdział 1 stanowi wprowadzenie, przedstawiając cel pracy, motywację powstania systemu oraz charakterystykę gier tekstowych RPG.

Rozdział 2 zawiera analizę istniejących rozwiązań: silników gier, narzędzi do tworzenia gier RPG oraz platform dystrybucji i formułuje założenia projektowe systemu.

Rozdział 3 obejmuje analizę systemu: wymagania funkcjonalne i нефункционалне, definicję aktorów, diagramy przypadków użycia i sekwencji oraz model danych.

Rozdział 4 przedstawia projekt systemu – architekturę, projekt bazy danych, interfejsu użytkownika, modułów oraz API.

Rozdział 5 opisuje implementację poszczególnych komponentów systemu z uwzględnieniem zastosowanych technologii.

Rozdział 6 omawia strategię i wyniki testowania systemu.

Rozdział 7 zawiera podsumowanie realizacji projektu, ograniczenia systemu oraz propozycje dalszego rozwoju.

2. Analiza problemu i istniejących rozwiązań

W niniejszym rozdziale przeprowadzono analizę problemu tworzenia tekstowych gier RPG oraz przegląd istniejących rozwiązań wykorzystywanych do ich projektowania i dystrybucji.

Omówiono charakterystykę gier tekstowych, dostępne silniki i narzędzia wykorzystywane przez twórców oraz platformy umożliwiające publikację gotowych projektów.

Celem analizy było zidentyfikowanie ograniczeń obecnych rozwiązań oraz określenie problemów, które powinien rozwiązywać projektowany system. Na podstawie przeprowadzonych obserwacji sformułowano również założenia projektowe wykorzystane w dalszych etapach pracy.

2.1. Architektura i logika systemowa gier tekstowych

Z perspektywy inżynierii oprogramowania, pomimo zredukowanej warstwy prezentacji, gry tekstowe RPG charakteryzują się rygorystyczną i złożoną architekturą wewnętrzną. Podczas gdy w produkcjach graficznych znaczna część zasobów obliczeniowych przeznaczana jest na renderowanie obrazu i symulację fizyki, w grach tekstowych kluczowym problemem technicznym jest asynchroniczne przetwarzanie i mutacja stanu gry (ang. *game state*) oraz spójna ewaluacja logiki biznesowej.

Podstawowym elementem takiego systemu jest model świata gry, który przechowuje informacje o aktualnym stanie rozgrywki. Obejmuje on między innymi aktualną lokalizację gracza w grafie pomieszczeń, zawartość ekwipunku, relacje między encjami oraz flagi zdarzeń warunkujących postęp fabularny. Każda akcja użytkownika powoduje zmianę tego stanu, co wpływa na dalszy przebieg gry.

Kolejnym istotnym komponentem jest mechanizm interpretacji działań gracza. W zależności od przyjętego podejścia może on przyjmować formę parsera poleceń lub systemu wyboru. Parser poleceń wymaga przetwarzania wprowadzonych przez użytkownika komend i ich mapowania na operacje systemowe, co wiąże się z koniecznością obsługi różnych wariantów składni. System wyboru upraszcza ten proces poprzez ograniczenie możliwych działań do zestawu zdefiniowanych opcji.

Krytycznym aspektem implementacyjnym gier tekstowych RPG jest również mechanizm utrwalania danych (ang. *data persistence*). Aplikacja musi zapewniać bezstratną serializację pełnego stanu rozgrywki do trwałego nośnika (np. w postaci zagnieżdżonych struktur JSON) oraz jego późniejszą deserializację. Mechanizm ten gwarantuje zachowanie ciągłości sesji i wymusza zaprojektowanie bezstanowych silników rozgrywki, zdolnych do odtworzenia dowolnego punktu w grze wyłącznie na podstawie dostarczonego pliku zapisu.

2.2. Narzędzia do tworzenia gier RPG

Istniejące silniki do tworzenia gier komputerowych mogą być również wykorzystane do tworzenia tekstowych gier RPG. Charakteryzują się one istotnymi ograniczeniami.

Silniki ogólne takie jak *Unity* [5] czy *Unreal Engine* [6] to zaawansowane narzędzia dla deweloperów. Są to kompleksowe rozwiązania wykorzystywane zarówno przez indywidualnych twórców, jak i profesjonalne studia. Oferują szeroki zakres funkcjonalności, w tym biblioteki do symulacji fizyki w przestrzeni 3D, czy systemy oświetlenia z technologią ray tracing. Nierzadko, w celu wytworzenia nawet najprostszej gry, wymagane jest napisanie kodu w języku C# (*Unity*) lub C++ (*Unreal Engine*).

Rozmiar instalacji tych silników może osiągać nawet dziesiątki gigabajtów (w zależności od komponentów) [7]. Gry wytworzone w nich bardzo szybko osiągają setki megabajtów, rosnąc przy dodaniu każdej nowej paczki zasobów. Przy wytwarzaniu tekstowych gier RPG, takie rozmiary plików znacząco przekraczają minimum wymagane do dostarczenia w pełni funkcjonalnej gry.

Zatem, ogólne silniki, takie jak *Unity*, czy *Unreal Engine*, nie są optymalnym rozwiązaniem dla osób tworzących tekstowe gry.

Na rynku można również znaleźć dedykowane rozwiązania do tworzenia gier RPG, takie jak *RPG Maker* [8]. *RPG Maker* jest narzędziem o zawężonym zakresie funkcjonalności, skupiającym się na grach 2D. Główny model jednak opiera się na podejściu graficznym - oczekuje się od użytkownika obrazków (ang. „sprites”) oraz podejścia „rzut z góry” (tak jak w grze *Star-dew Valley*) lub „rzut z boku” (np. gry typu *Flappy Bird*). Próba wytworzenia tekstowej gry RPG wymaga implementacji znaczącej części logiki systemu od podstaw i ponadpodstawowej znajomości wybranego języka programowania. Te utrudnienia przemawiają przeciwko użyciu tego programu do wytwarzania takich gier.

Najbardziej odpowiednim doбором silnika wydają się te stworzone do projektowania gier tekstowych. Należą do nich takie programy jak *TADS*, *Inform 7* [9] lub *Twine* [10]. Mimo specyficznego zastosowania, często wymagają jednak kodu w narzuconym z góry (czasem własnym) języku programowania. Dodatkowo, złożoność interfejsu użytkownika może wydawać się przytłaczająca dla laików.

Zatem istniejące na rynku rozwiązania umożliwiające tworzenie gier mają spore wady, a ich użytkowanie może stanowić istotną barierę wejścia dla osób bez doświadczenia w pisaniu kodu lub obsłudze technicznych programów.

2.3. Platformy, które zajmują się dystrybucją gier stworzonych przez użytkowników

Obecnie twórcy gier komputerowych mają do dyspozycji szereg platform umożliwiających dystrybucję własnych projektów. Różnią się one modelem biznesowym, progiem wejścia oraz docelową grupą odbiorców, co znacząco wpływa na proces publikacji.

Platformy głównego nurtu: Najpopularniejszym cyfrowym sklepem z grami pozostaje *Steam* [11], w którym można znaleźć zarówno wysokobudżetowe produkcje (np. *Cyberpunk 2077* [12]), jak i gry twórców niezależnych. Publikacja w tym serwisie wiąże się jednak z istotnymi barierami wejścia – wymaga uiszczenia bezzwrotnej opłaty publikacyjnej oraz przejścia wieloetapowego procesu weryfikacji projektów, a sama platforma pobiera do 30% prowizji od wygenerowanych przychodów [13]. Głównym konkurentem dla platformy *Steam* jest *Epic Games Store* [14]. Przyciąga on deweloperów znacznie niższą marżą, wynoszącą 12% [15]. Choć początkowo platforma ta stosowała wysoce restrykcyjną politykę kuratorską, z czasem udostępniła narzędzia do samodzielnej publikacji (ang. *self-publishing tools*) dla szerszego grona twórców [16]. Warto również wymienić serwis *GOG.com* [17], skupiający się na dystrybucji gier pozbawionych zabezpieczeń DRM (ang. *Digital Rights Management*). Ze względu na rygorystyczny proces selekcji tytułów, platforma ta rzadko stanowi miejsce publikacji dla mniejszych, hobbystycznych projektów.

Platformy otwarte dla twórców niezależnych: Serwisy takie jak *itch.io* [18] oraz *Game Jolt* [19] są z założenia skoncentrowane na projektach o mniejszej skali, prototypach oraz grach typu „indie”. Wyróżniają się one brakiem opłat za publikację oraz dużą elastycznością w ustalaniu modeli cenowych, w tym opcją dobrowolnych wpłat od graczy (ang. *pay-what-you-want*). Co kluczowe z punktu widzenia projektów tekstowych, platformy te umożliwiają osadzanie gier bezpośrednio w oknie przeglądarki z wykorzystaniem technologii *HTML5* [20]. Należy jednak zaznaczyć, że mimo wsparcia dla tego formatu, serwisy te służą wyłącznie do dystrybucji i nie dostarczają wbudowanych, autorskich narzędzi do projektowania i kompilacji gier.

Sklepy z oprogramowaniem mobilnym: Platformy takie jak *Google Play* [21] oraz *App Store* [22] stanowią naturalne i atrakcyjne środowisko dla gier tekstowych ze względu na komfort czytania treści na ekranach urządzeń przenośnych. Dystrybucja w nich narzuca jednak na twórcę konieczność samodzielnego przygotowania, dostosowania i skompilowania aplikacji pod konkretny system operacyjny, a także wymusza opłacenie kont deweloperskich oraz dostosowanie się do rygorystycznych wytycznych technicznych [23].

Platformy dedykowane grom tekstowym: W niszowym sektorze narracji interaktywnej funkcjonują wyspecjalizowane wydawnictwa i serwisy, takie jak *Choice of Games* [24] czy *TextAdventures.co.uk* [25]. Często wymuszają one jednak na twórcach stosowanie własnych, dedykowanych silników i języków skryptowych, co ponownie wiąże się z koniecznością przyswojenia wiedzy technicznej.

Jak wynika z powyższej analizy dostępnych rozwiązań rynkowych, mimo bogatej oferty żadna z platform ogólnego przeznaczenia nie oferuje zintegrowanego środowiska, które łączyłoby intuicyjny edytor typu *no-code* z mechanizmem natychmiastowej, zautomatyzowanej publikacji w obrębie jednego spójnego systemu.

2.4. Wnioski z analizy istniejących rozwiązań

Na podstawie przeprowadzonej analizy można stwierdzić, że istnieje szeroki zakres narzędzi oraz platform umożliwiających tworzenie i dystrybucję gier komputerowych. Rozwiązania te oferują rozbudowane funkcjonalności oraz wsparcie dla różnych typów projektów, od gier prostych po zaawansowane produkcje komercyjne.

Jednocześnie zidentyfikowano szereg istotnych ograniczeń. W przypadku ogólnych silników gier, takich jak *Unity* czy *Unreal Engine*, problemem jest wysoki poziom złożoności oraz konieczność posiadania umiejętności programistycznych. Dedykowane narzędzia do tworzenia gier RPG często opierają się na modelu graficznym, który nie odpowiada specyfice gier tekstowych.

Narzędzia przeznaczone bezpośrednio do tworzenia gier tekstowych, mimo swojej specjalizacji, nadal wymagają znajomości języków programowania lub specyficznych składni, co ogranicza ich dostępność dla osób bez doświadczenia technicznego. Dodatkowo, ich interfejsy użytkownika mogą być złożone i nieintuicyjne.

W zakresie dystrybucji gier, istniejące platformy umożliwiają publikację projektów tworzonych przez użytkowników, jednak nie oferują integracji procesu tworzenia z publikacją. Twórca jest zobowiązany do korzystania z zewnętrznych narzędzi oraz samodzielnego przygotowania gotowego produktu.

Na podstawie powyższych obserwacji można wskazać kluczowe problemy, takie jak wysoki próg wejścia, brak narzędzi typu no-code dedykowanych grom tekstowym oraz brak zintegrowanych rozwiązań łączących proces tworzenia i dystrybucji.

2.5. Założenia projektowe systemu

Na podstawie zidentyfikowanych problemów sformułowano założenia projektowe opracowanego systemu.

W celu obniżenia progu wejścia przyjęto założenie, że system nie wymaga znajomości języków programowania i umożliwia tworzenie gier w modelu no-code.

W odpowiedzi na brak integracji pomiędzy tworzeniem a dystrybucją, zaprojektowano rozwiązanie łączące edytor gier z platformą ich publikacji i udostępniania.

Ze względu na ograniczenia istniejących narzędzi desktopowych przyjęto, że system będzie dostępny w pełni przez przeglądarkę internetową, bez konieczności instalacji dodatkowego oprogramowania.

W celu zachowania klasycznego charakteru gier tekstowych zrezygnowano z implementacji zasobów multimedialnych, takich jak grafika, dźwięk czy wideo. Główny nacisk położono na tekstową warstwę prezentacji oraz wielowymiarowy model interakcji użytkownika ze środowiskiem gry. Założono, że mechanika ta będzie obejmować nawigację przestrzenną pomiędzy połączonymi grafowo lokacjami, zarządzanie ekwipunkiem (obejmujące podnoszenie i stosowanie przedmiotów) oraz turowy system walki z przeciwnikami.

Ponadto, w ramach wzbogacenia warstwy narracyjnej i eksploracyjnej, przewidziano implementację interaktywnych elementów otoczenia, takich jak skrzynie, a także system rozbudowanych, deterministycznych drzew konwersacji, umożliwiające wielowątkową komunikację z postaciami niezależnymi (NPC).

Dodatkowo przyjęto założenie implementacji mechanizmu zapisu i odczytu stanu gry, umożliwiającego kontynuowanie rozgrywki.

3. Analiza systemu

Na podstawie założeń projektowych przedstawionych w poprzednim rozdziale przeprowadzono analizę projektowanego systemu *Digital Dungeons*. W ramach analizy określono wymagania funkcjonalne i нефункционалне, zidentyfikowano aktorów systemu oraz opracowano modele opisujące sposób działania poszczególnych mechanizmów.

W rozdziale przedstawiono również diagram przypadków użycia, scenariusze interakcji użytkownika z systemem, diagramy sekwencji oraz model danych wykorzystywany przez aplikację. Elementy te stanowią podstawę do dalszego etapu projektowania i implementacji systemu.

3.1. Wymagania funkcjonalne

Na podstawie założeń projektowych sformułowanych w rozdziale 2 opracowano wymagania funkcjonalne systemu. Opisują one zestaw operacji, które system powinien udostępniać swoim użytkownikom.

- WF-01: System umożliwia rejestrację nowego konta użytkownika na podstawie podanego loginu, adresu e-mail i hasła.
- WF-02: System umożliwia logowanie do istniejącego konta użytkownika.
- WF-03: System umożliwia tworzenie nowych gier tekstowych przy użyciu graficznego edytora, bez konieczności znajomości programowania.
- WF-04: System umożliwia definiowanie pomieszczeń gry wraz z ich nazwą i opisem tekstowym.
- WF-05: System umożliwia dodawanie przedmiotów do pomieszczeń oraz definiowanie ich właściwości (nazwa, opis). Podczas rozgrywki gracz może podnosić, upuszczać i używać przedmioty.
- WF-06: System umożliwia tworzenie postaci niezależnych (NPC) i przypisywanie im drzew konwersacji.
- WF-07: System automatycznie tworzy przejścia kierunkowe pomiędzy pomieszczeniami sąsiadującymi na siatce edytora.
- WF-08: System umożliwia publikację ukończonej gry w serwisie marketplace.
- WF-09: System umożliwia przeglądanie opublikowanych gier w marketplace z możliwością wyszukiwania i filtrowania.
- WF-10: System umożliwia rozgrywkę za pomocą wpisywanych komend tekstowych.
- WF-11: System umożliwia zapis stanu rozgrywki oraz jej późniejsze wznowienie.
- WF-12: System umożliwia dodawanie komentarzy do gier oraz ich polubienie.
- WF-13: System umożliwia zarządzanie profilem użytkownika, w tym zmianę danych konta.
- WF-14: System umożliwia edytowanie i usuwanie własnych gier przez ich twórców.

3.2. Wymagania нефункционалне

Wymagania нефункционалне opisują ograniczenia i cechy jakościowe systemu, niezwiązane bezpośrednio z jego funkcjonalnością.

- WNF-01 (Dostępność): System działa w całości w przeglądarce internetowej, bez konieczności instalowania dodatkowego oprogramowania. Jest kompatybilny ze współczesnymi przeglądarkami obsługującymi standard *ECMAScript 2020*.
- WNF-02 (Bezpieczeństwo): Uwierzytelnianie opiera się na tokenach JWT (ang. JSON Web Token) przechowywanych w ciasteczkach `httpOnly`, ustawianych przez serwer po pomyślnym logowaniu lub rejestracji. Ciasteczka są automatycznie dołączane do każdego żądania HTTP i nie są dostępne dla kodu po stronie klienta, co chroni przed atakami XSS (ang. Cross-Site Scripting). Hasła przechowywane są w formie zahasowanej z użyciem algorytmu *bcrypt*. Endpointy rejestracji i logowania chronione są przez mechanizm ograniczania częstotliwości żądań (ang. *rate limiting*) oparty na adresie IP. Serwer korzysta z biblioteki *Helmet*, która ustawia bezpieczne nagłówki HTTP

(m.in. X-Content-Type-Options, X-Frame-Options). Dostęp do chronionych zasobów jest weryfikowany po stronie serwera API.

- WNF-03 (Ograniczenie mediów): System nie obsługuje elementów graficznych, dźwiękowych ani wideo. Narracja i interakcja opierają się wyłącznie na tekście.
- WNF-04 (Wydajność): Serwer API powinien odpowiadać na zapytania w czasie nieprzekraczającym 2 sekund. Rozmiar danych gry przesyłanych przez API jest ograniczony do 10 MB.
- WNF-05 (Niezawodność zapisu): Stan rozgrywki jest zapisywany przy wyjściu z gry za pomocą komendy QUIT/EXIT lub przycisku "SAVE & QUIT", co umożliwia wznowienie rozgrywki od ostatniego punktu wyjścia.
- WNF-06 (Intuicyjność): Edytor gier jest zaprojektowany w modelu no-code, umożliwiając tworzenie gier osobom bez doświadczenia programistycznego.

3.3. Aktorzy systemu

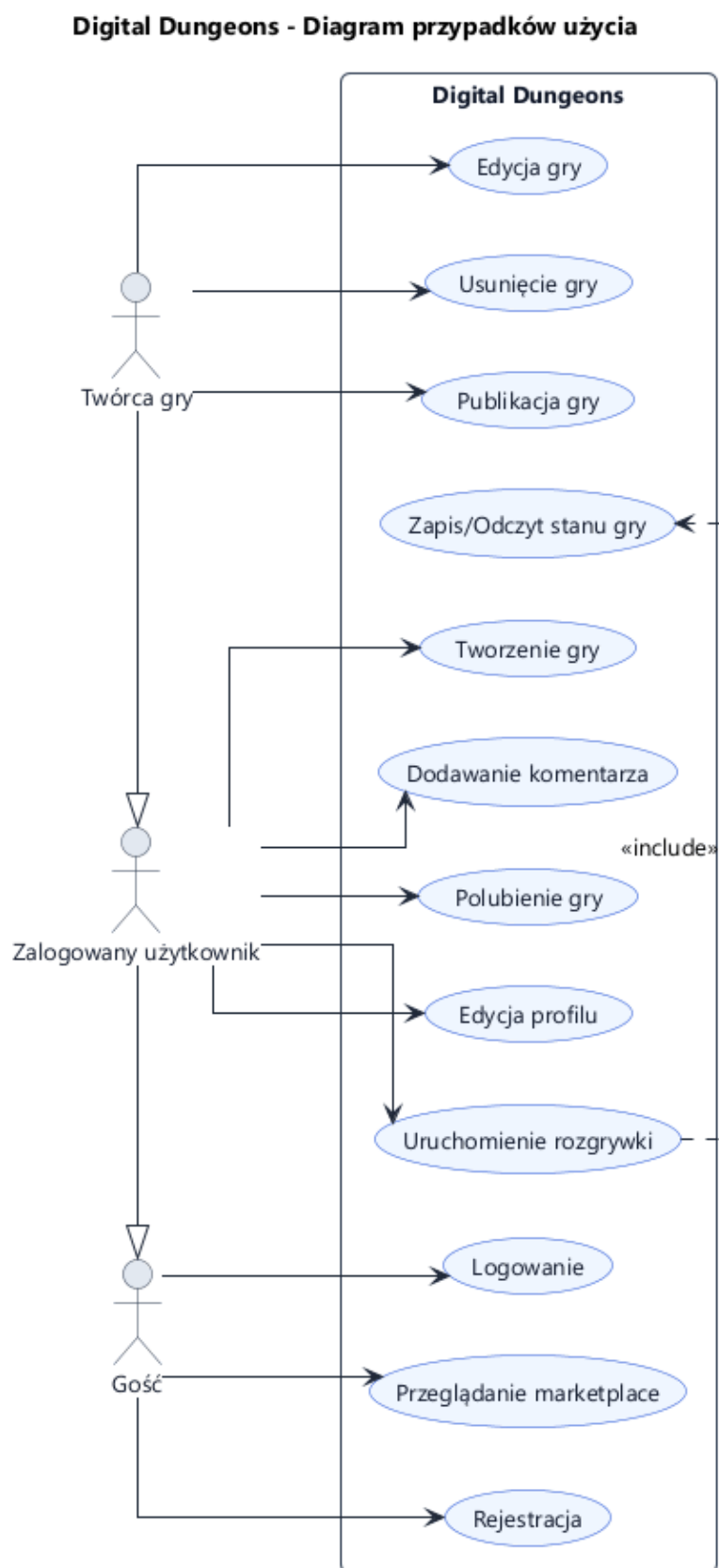
W systemie *Digital Dungeons* zidentyfikowano trzech aktorów.

Pierwszym z nich jest *Gość*, czyli użytkownik niezalogowany. Ma on dostęp do przeglądania opublikowanych gier w marketplace, przeglądania ich szczegółów oraz uruchamiania rozgrywki. Funkcje wymagające konta (polubienia, komentarze, zapis stanu gry) są dla niego niedostępne.

Kolejnym aktorem jest *Zalogowany użytkownik*, dysponujący pełną funkcjonalnością systemu: może grać, tworzyć i publikować gry, dodawać komentarze i polubienia oraz zarządzać swoim profilem.

Twórca gry to rola przysługująca zalogowanemu użytkownikowi względem jego własnych gier. Jako jedyny może je edytować i usuwać. Każdy użytkownik staje się twórcą w momencie utworzenia gry.

3.4. Diagram przypadków użycia



Rys. 2. Diagram przypadków użycia

3.5. Opis scenariuszy przypadków użycia

Poniżej podano szczegółowe scenariusze dla wybranych przypadków użycia systemu.

UC-01: Rejestracja użytkownika

Pole	Opis
Identyfikator	UC-01
Nazwa	Rejestracja użytkownika
Aktorzy	Gość
Warunek wstępny	Użytkownik nie posiada konta lub nie jest zalogowany
Główny przepływ	1. Użytkownik przechodzi do formularza rejestracji. 2. Użytkownik wprowadza login, adres e-mail i hasło. 3. System weryfikuje poprawność i unikalność danych. 4. System tworzy nowe konto i automatycznie loguje użytkownika. 5. Użytkownik zostaje przekierowany na stronę główną.
Przepływ alternatywny	3a. Podany adres e-mail lub login jest już zajęty. System wyświetla komunikat błędu i prosi o podanie innych danych.
Warunek końcowy	W systemie zostało utworzone nowe konto użytkownika.

UC-02: Logowanie

Pole	Opis
Identyfikator	UC-02
Nazwa	Logowanie
Aktorzy	Gość
Warunek wstępny	Użytkownik posiada konto w systemie
Główny przepływ	1. Użytkownik przechodzi do formularza logowania. 2. Użytkownik wprowadza adres e-mail i hasło. 3. System weryfikuje zgodność danych z zapisem w bazie. 4. System generuje token JWT i ustawia go jako ciasteczko httpOnly. 5. Użytkownik zostaje przekierowany na stronę główną jako zalogowany.
Przepływ alternatywny	3a. Dane logowania są nieprawidłowe. System wyświetla komunikat błędu, użytkownik pozostaje niezalogowany.
Warunek końcowy	Użytkownik jest zalogowany i posiada aktywną sesję.

UC-03: Tworzenie gry

Pole	Opis
Identyfikator	UC-03
Nazwa	Tworzenie nowej gry
Aktorzy	Zalogowany użytkownik
Warunek wstępny	Użytkownik jest zalogowany
Główny przepływ	1. Użytkownik przechodzi do edytora gier. 2. Użytkownik wprowadza tytuł i opis gry. 3. Użytkownik dodaje pomieszczenia na kanwie graficznej edytora. 4. Użytkownik definiuje właściwości pomieszczeń, przedmiotów i NPC. 5. Użytkownik zapisuje grę. 6. System zapisuje definicję gry w bazie danych w formacie JSON.
Przepływ alternatywny	6a. Gra nie zawiera żadnych pomieszczeń. System informuje użytkownika o braku pomieszczeń i wstrzymuje zapis.
Warunek końcowy	Gra jest zapisana w systemie jako nieopublikowana i widoczna tylko dla jej twórcy.

UC-04: Rozgrywka

Pole	Opis
Identyfikator	UC-04
Nazwa	Prowadzenie rozgrywki
Aktorzy	Zalogowany użytkownik, Gość
Warunek wstępny	Wybrana gra jest opublikowana lub należy do zalogowanego użytkownika. Gość może uruchomić grę, lecz postęp nie zostanie zapisany
Główny przepływ	1. Użytkownik wybiera grę z marketplace lub własnej listy. 2. System wczytuje definicję gry i poprzedni stan rozgrywki (jeśli istnieje). 3. Silnik rozgrywki inicjalizuje stan gry. 4. System wyświetla opis pomieszczenia startowego. 5. Użytkownik wprowadza komendę tekstową. 6. Silnik interpretuje komendę i aktualizuje stan gry. 7. System wyświetla odpowiedź tekstową. 8. Kroki 5–7 są powtarzane aż do zakończenia gry lub opuszczenia jej przez gracza. 9. System automatycznie zapisuje stan rozgrywki.
Przepływ alternatywny	6a. Komenda jest nierozpoznana. System wyświetla komunikat o nieznannej komendzie, stan gry nie ulega zmianie.
Warunek końcowy	Stan rozgrywki jest zapisany w bazie danych i może być wznowiony w przyszłości.

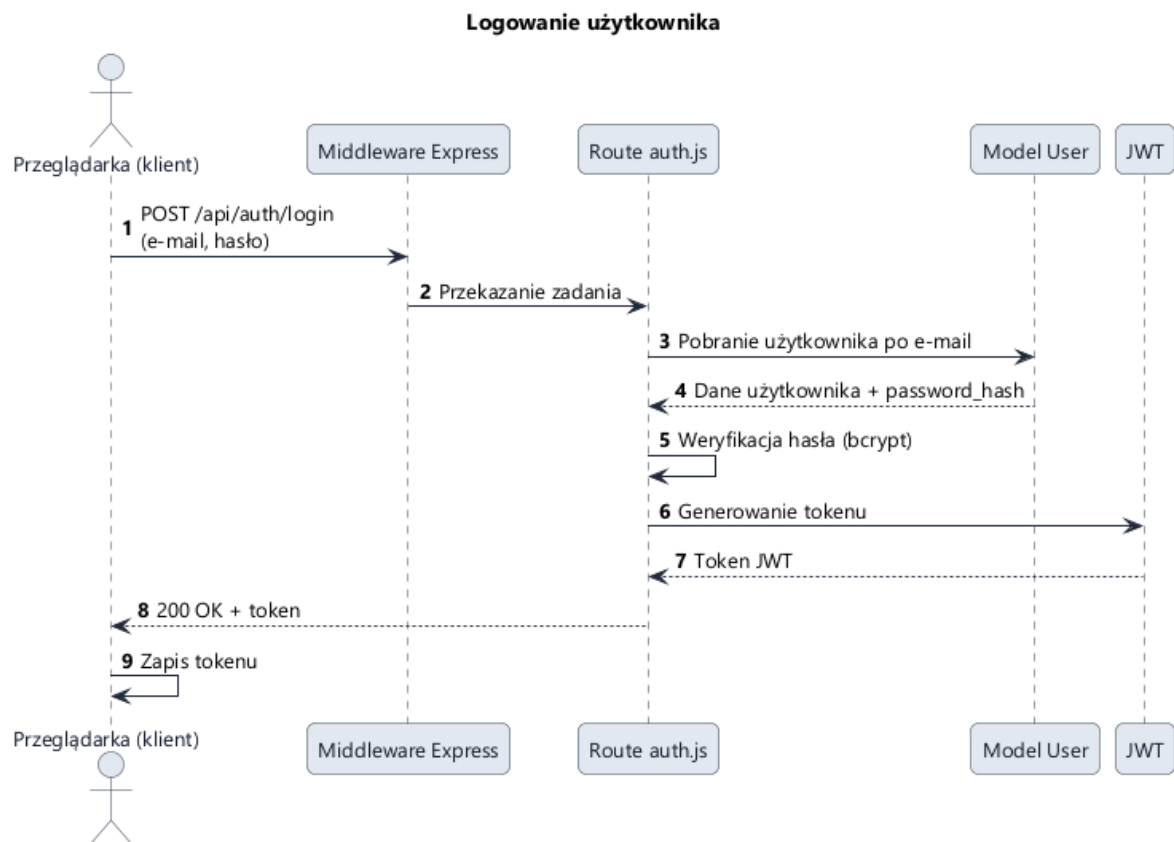
UC-05: Publikacja gry

Pole	Opis
Identyfikator	UC-05
Nazwa	Publikacja gry w marketplace
Aktorzy	Twórca gry
Warunek wstępny	Użytkownik jest zalogowany; gra jest zapisana w systemie
Główny przepływ	1. Użytkownik przechodzi do zarządzania swoją grą. 2. Użytkownik wybiera opcję publikacji. 3. System zmienia status gry na opublikowany. 4. Gra staje się widoczna w serwisie marketplace dla wszystkich użytkowników.
Przepływ alternatywny	brak
Warunek końcowy	Gra jest opublikowana i dostępna publicznie w marketplace.

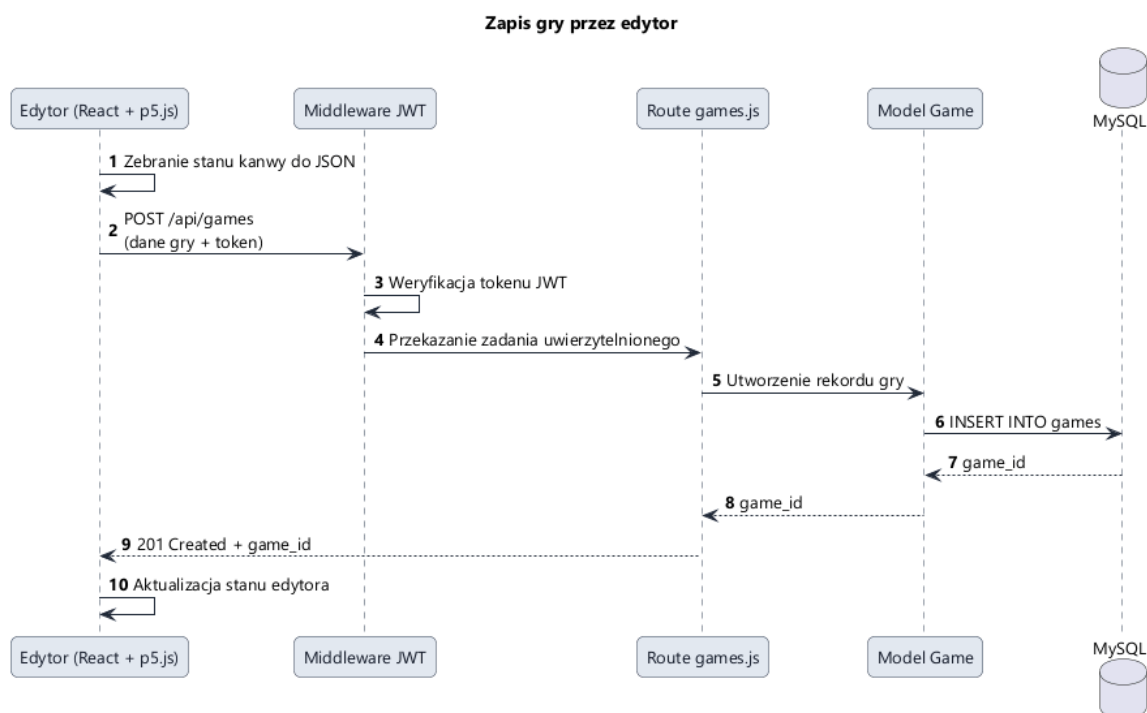
UC-06: Dodawanie komentarza

Pole	Opis
Identyfikator	UC-06
Nazwa	Dodawanie komentarza do gry
Aktorzy	Zalogowany użytkownik
Warunek wstępny	Użytkownik jest zalogowany; wybrana gra jest opublikowana
Główny przepływ	1. Użytkownik przechodzi na stronę gry w marketplace. 2. Użytkownik wpisuje treść komentarza w formularzu. 3. Użytkownik zatwierdza komentarz. 4. System zapisuje komentarz powiązany z grą i użytkownikiem. 5. Komentarz staje się widoczny na stronie gry dla innych użytkowników.
Przepływ alternatywny	3a. Treść komentarza jest pusta. System nie zapisuje komentarza. Przycisk wysłania jest nieaktywny dla pustego pola.
Warunek końcowy	Komentarz jest zapisany w bazie danych i widoczny publicznie.

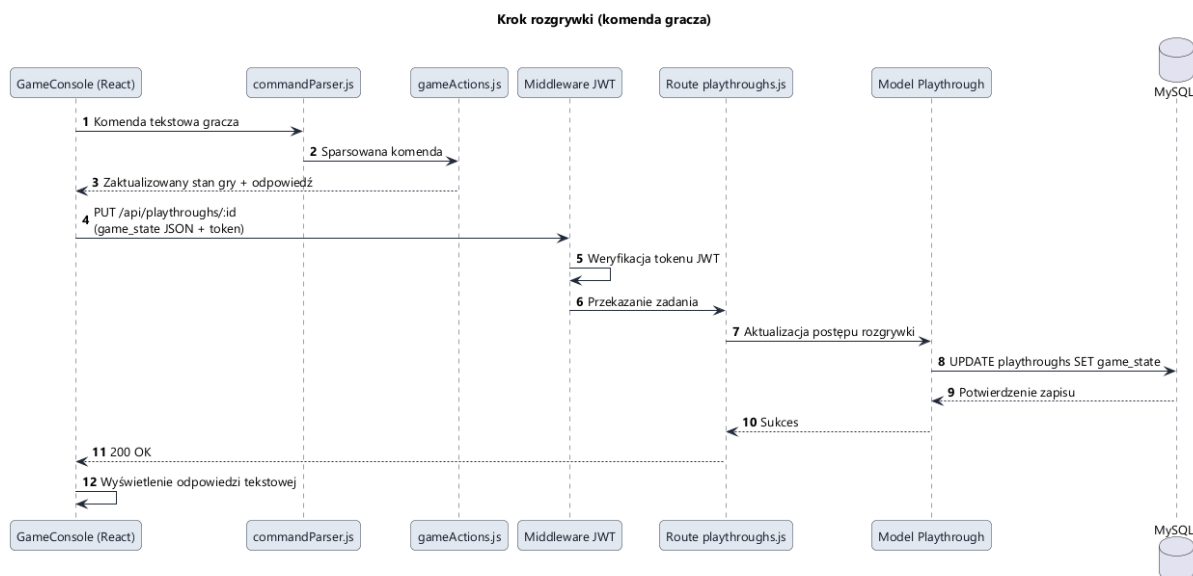
3.6. Diagramy sekwencji



Rys. 3. Diagram sekwencji – logowanie użytkownika



Rys. 4. Diagram sekwencji – zapis gry przez edytor



Rys. 5. Diagram sekwencji – krok rozgrywki

3.7. Model danych systemu

Model danych systemu *Digital Dungeons* opiera się na relacyjnej bazie danych *MySQL/MariaDB*. Dane przechowywane są w pięciu tabelach obejmujących użytkowników, gry, sesje rozgrywki oraz interakcje społecznościowe.

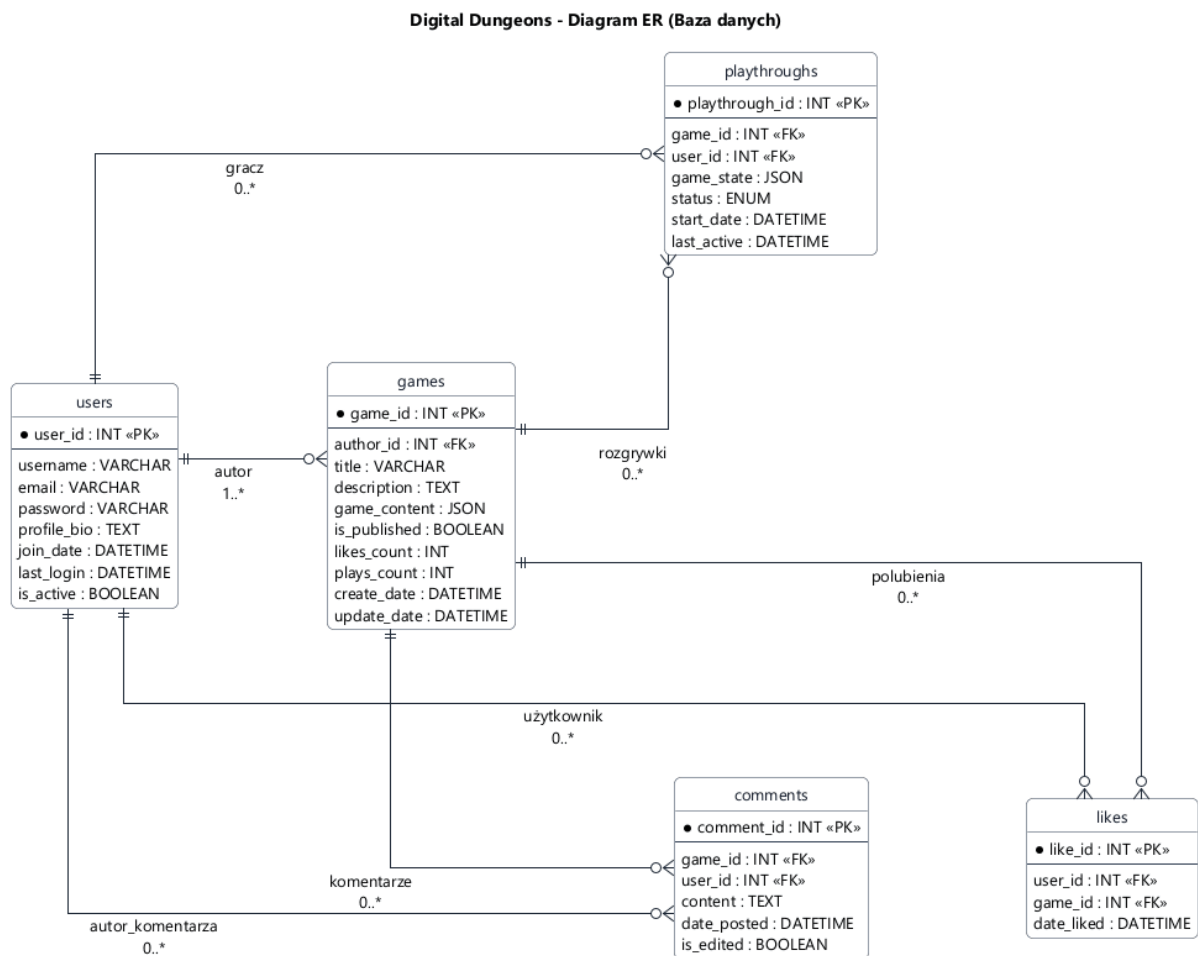
Tabela *users* przechowuje dane kont użytkowników: identyfikator (*user_id*), nazwę użytkownika (*username*), adres e-mail, zahasowane hasło (*password*), opis profilu (*profile_bio*) oraz znaczniki czasu rejestracji (*join_date*) i ostatniego logowania (*last_login*). Pole *is_active* umożliwia dezaktywację konta.

Tabela *games* zawiera metadane gier oraz ich pełną definicję. Pole *game_content* przechowuje strukturę gry w formacie JSON, zawierającą listę pomieszczeń z ich opisami, przedmiotami, postaciami NPC oraz identyfikatorem pomieszczenia startowego. Pole *is_published* określa widoczność gry w marketplace. Pola *likes_count* i *plays_count* przechowują zdenormalizowane liczniki polubień i uruchomień. Klucz obcy *author_id* odnosi się do tabeli *users*.

Tabela *playthroughs* rejestruje sesje rozgrywki. Pole *game_state* przechowuje bieżący stan gry w formacie JSON, obejmującym aktualny identyfikator pomieszczenia gracza (*currentRoomId*), zawartość ekwipunku (*inventory*) oraz stany poszczególnych pomieszczeń (*roomStates*) – informacje o dostępnych w nich przedmiotach, pokonanych wrogach, stanie skrzyń i postępie konwersacji. Pole *status* przyjmuje wartości *in_progress*, *completed* lub *abandoned*. Klucze obce *game_id* i *user_id* wiążą sesję z konkretną grą i jej graczem.

Tabela *likes* przechowuje pary (*user_id*, *game_id*) ze znacznikiem czasu, realizując funkcję polubień. Kombinacja obu kluczy obcych jest unikalna, co uniemożliwia wielokrotne polubienie tej samej gry przez jednego użytkownika.

Tabela *comments* przechowuje komentarze użytkowników do gier. Każdy komentarz jest powiązany z grą (*game_id*) i autorem (*user_id*) oraz zawiera treść tekstową i znacznik czasu dodania. Pole *is_edited* oznacza, czy komentarz był edytowany po opublikowaniu.



Rys. 6. Diagram ER

Strukturę danych JSON przechowywanych w polu *game_content* tabeli *games* ilustruje poniższy przykład:

```

{
  "rooms": [
    {
      "id": "0,0",
      "gx": 0,
      "gy": 0,
      "meta": {
        "name": "Wejście do jaskini",
        "description": "Stoisz u wejścia do mrocznej jaskini.",
        "isStart": true,
        "entities": ["goblin_scout"],
        "items": ["torch"]
      }
    }
  ]
}
  
```

```

    },
    {
      "id": "1,0",
      "gx": 1,
      "gy": 0,
      "meta": {
        "name": "Komnata skarbca",
        "description": "Ciemna komnata z zamkniętą skrzynią.",
        "hasChest": true,
        "chestGuardian": "goblin_scout",
        "chestRequiresKey": "rusty_key",
        "chestContents": ["gold_coin"]
      }
    }
  ],
  "selected": "0,0",
  "globalMeta": {
    "gameName": "Jaskinia goblinów",
    "gameDescription": "Eksploruj jaskinie pełne goblinów.",
    "tags": ["cave", "goblins"],
    "entities": [
      {
        "id": "goblin_scout",
        "name": "Goblin Zwiadowca",
        "type": "monster",
        "hostile": true,
        "drops": ["rusty_key"]
      }
    ],
    "items": [
      {
        "id": "torch",
        "name": "Pochodnia",
        "description": "Oświecła ciemne korytarze."
      },
      {
        "id": "rusty_key",
        "name": "Zardzewiały klucz",
        "description": "Klucz upuszczony przez goblina."
      },
      {
        "id": "gold_coin",
        "name": "Złota moneta",
        "description": "Cenna złota moneta."
      }
    ]
  }
}

```

Pomieszczenia sąsiadujące na siatce (różniące się o 1 w osi *gx* lub *gy*) są automatycznie połączone przejściami kierunkowymi. Pole *game_state* tabeli *playthroughs* przechowuje dynamiczny stan rozgrywki, aktualizowany po każdej akcji gracza:

```
{
  "currentRoomId": "1,0",
  "inventory": ["torch", "rusty_key"],
  "roomStates": {
    "0,0": {
      "items": [],
      "defeatedEntities": ["goblin_scout"],
      "guardiansDefeated": ["goblin_scout"],
      "chestOpened": false,
      "visitedConversations": {}
    }
  }
}
```

4. Projekt systemu

W niniejszym rozdziale przedstawiono projekt systemu *Digital Dungeons* z perspektywy jego architektury, modelu danych, interfejsu użytkownika, podziału na moduły oraz udostępnianego API. Opis ten stanowi pomost pomiędzy analizą problemu a etapem implementacji, ponieważ porządkuje sposób współpracy poszczególnych komponentów aplikacji.

4.1. Architektura systemu

Digital Dungeons został zaprojektowany jako aplikacja webowa o architekturze trójwarstwowej. Warstwa prezentacji działa w przeglądarce i zaimplementowana jest w *Next.js* z wykorzystaniem *React*. Warstwa logiki biznesowej realizowana jest przez serwer API oparty na *Express.js* i obsługuje autoryzację, walidację oraz reguły biznesowe. Warstwa danych opiera się na relacyjnej bazie *MySQL/MariaDB*, w której przechowywane są konta użytkowników, definicje gier, sesje rozgrywek oraz interakcje społeczne. Taki podział umożliwia niezależne rozwijanie poszczególnych warstw i zachowanie przejrzystej separacji odpowiedzialności.

Warstwa klienta odpowiada za renderowanie strony głównej, edytora gier, marketplace, profilu użytkownika oraz ekranu rozgrywki. W części edycyjnej wykorzystano bibliotekę *p5.js* do renderowania płótna graficznego, na którym umieszczane są pokoje, obiekty i powiązania pomiędzy elementami gry. Mechanizm komunikacji między *p5.js* a komponentami *React* oparto na globalnym obiekcie pośredniczącym oraz zdarzeniach DOM, dzięki czemu stan edytora może być synchronizowany z panelem bocznym i formularzami właściwości.

Warstwa serwerowa udostępnia zestaw endpointów REST, obsługuje walidację danych wejściowych, uwierzytelnianie oparte na tokenach JWT oraz dostęp do bazy danych *MySQL/MariaDB*. Po stronie serwera wydzielono osobne pliki routingu dla autoryzacji, gier,

użytkowników, polubień, komentarzy i sesji rozgrywki. Dzięki temu logika aplikacji jest rozdzielona zgodnie z odpowiedzialnościami poszczególnych obszarów systemu.

Architektura obejmuje również komponenty pomocnicze, takie jak warstwa przechowywania danych w relacyjnej bazie, mechanizmy kontroli dostępu oraz automatyczne zapisywanie stanu rozgrywki. Całość działa w modelu klient-serwer, a komunikacja odbywa się za pomocą żądań HTTP wysyłanych z frontendu do backendu.

4.2. Projekt bazy danych

Projekt bazy danych został oparty na relacyjnej strukturze *MySQL/MariaDB* i obejmuje pięć głównych tabel: *users*, *games*, *playthroughs*, *likes* oraz *comments*. Taki model dobrze odzwierciedla logikę serwisu, ponieważ oddziela dane użytkowników, definicje gier, historię rozgrywek i interakcje społeczne.

Tabela *users* przechowuje dane kont użytkowników, w tym nazwę użytkownika, adres e-mail, hasło w postaci zahasowanej, biogram, datę rejestracji, datę ostatniego logowania oraz znacznik aktywności konta. Na poziomie modelu aplikacji tabela ta odpowiada za rejestrację, logowanie, aktualizację profilu oraz pobieranie informacji o zalogowanym użytkowniku.

Tabela *games* zawiera podstawowe informacje o grach oraz ich pełną definicję. Oprócz identyfikatora, tytułu, opisu, dat utworzenia i aktualizacji przechowuje ona pole *game_content* zapisane w formacie JSON. W tym polu znajduje się struktura świata gry, obejmująca listę pokojów, metadane globalne, zdefiniowane przedmioty, encje oraz dane potrzebne do uruchomienia rozgrywki. Tabela zawiera również pola *likes_count* i *plays_count* - liczniki polubień i uruchomień, umożliwiające efektywne sortowanie w marketplace bez konieczności wykonywania kosztownych zapytań agregujących. Rekord gry jest powiązany z tabelą *users* przez klucz obcy *author_id*.

Tabela *playthroughs* przechowuje stan poszczególnych sesji rozgrywki. Zawiera ona identyfikatory gry i użytkownika, datę rozpoczęcia, datę ostatniej aktywności, bieżący stan gry w polu *game_state* oraz status sesji. Wartość status określa, czy rozgrywka jest w toku, zakończona czy porzucona. Rozwiązanie to umożliwia wznawianie gry w późniejszym czasie bez utraty postępu.

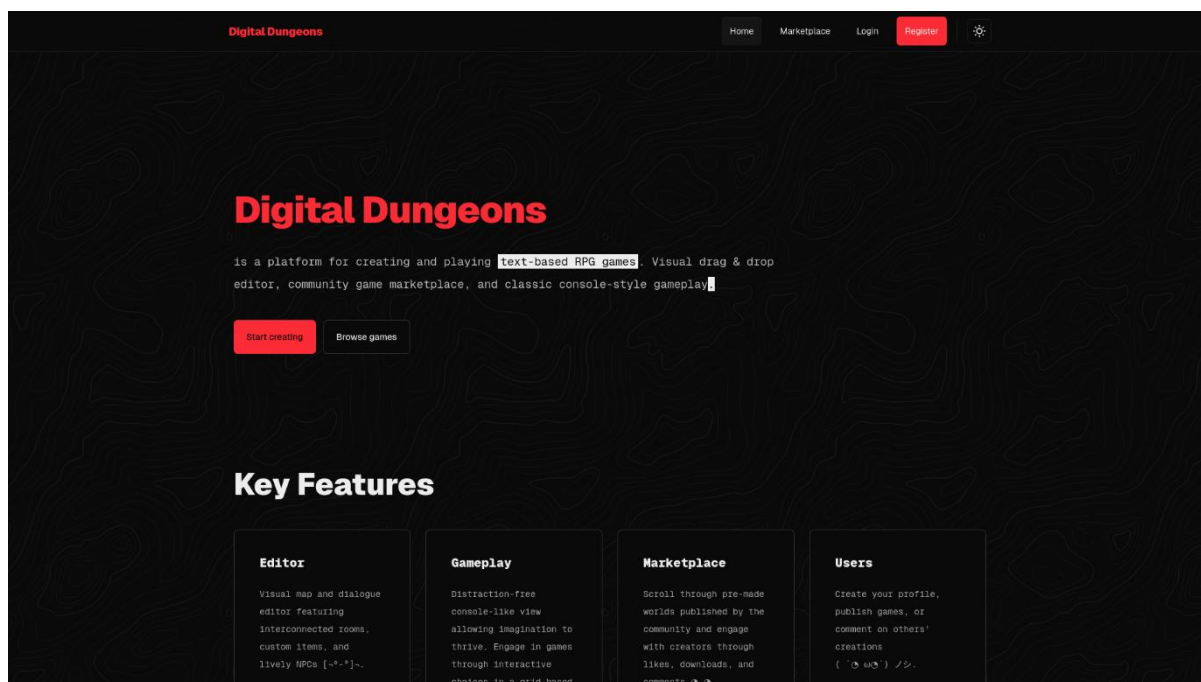
Tabela *likes* realizuje mechanizm polubień. Każdy rekord reprezentuje relację pomiędzy użytkownikiem a grą oraz przechowuje datę wystawienia polubienia. Unikalność pary *user_id* i *game_id* zapobiega wielokrotnemu polubieniu tej samej gry przez jednego użytkownika.

Tabela *comments* przechowuje komentarze dodawane do gier. Każdy wpis jest powiązany z autorem oraz konkretną grą, a oprócz treści zawiera znacznik czasu dodania i informację czy komentarz był edytowany. Dzięki temu możliwa jest zarówno publiczna dyskusja wokół gry, jak i późniejsza edycja treści przez autora komentarza.

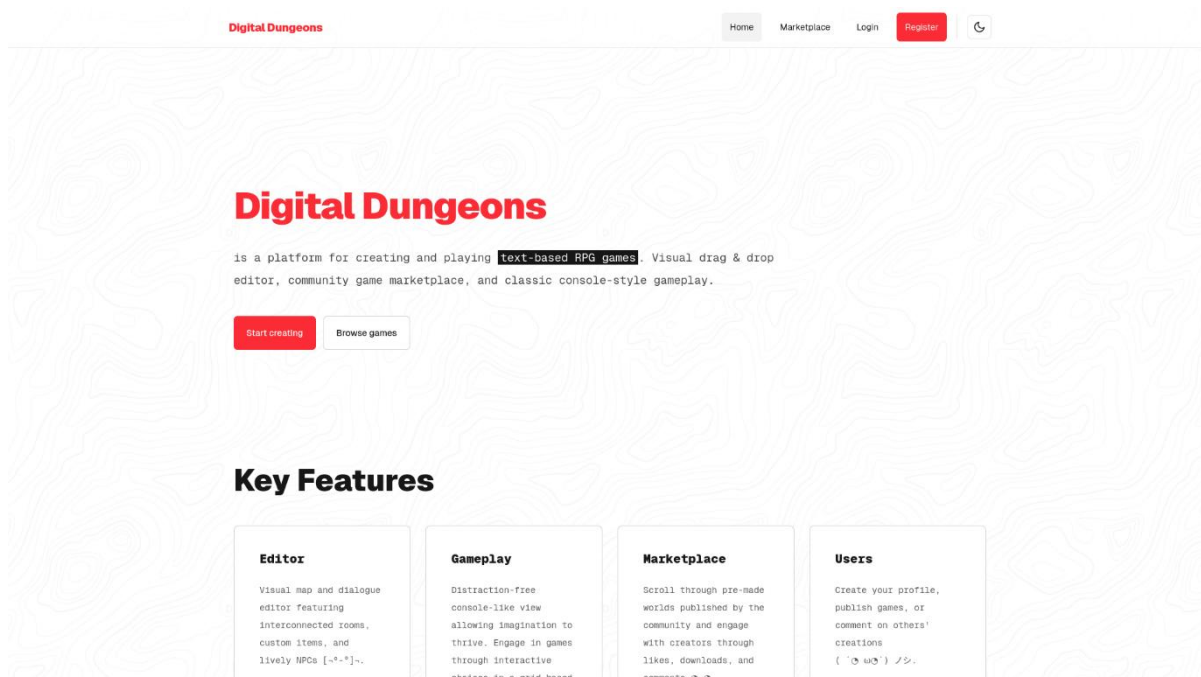
Układ relacji w bazie danych wspiera wszystkie podstawowe procesy serwisu: rejestrację i logowanie, publikację gier, ich przeglądanie, uruchamianie rozgrywki, zapisywanie postępu oraz interakcje społecznościowe. Zastosowanie typu JSON w polach *game_content* i *game_state* pozwala na elastyczne przechowywanie struktury świata gry i bieżącego stanu sesji bez konieczności rozbijania tych danych na zbyt dużą liczbę dodatkowych tabel.

4.3. Projekt interfejsu użytkownika

Interfejs użytkownika został zaprojektowany jako zestaw wyspecjalizowanych ekranów odpowiadających głównym scenariuszom użycia systemu. Strona główna pełni funkcję wprowadzającą i prezentuje podstawowe możliwości platformy. Z poziomu strony głównej użytkownik może przejść do edytora gier lub marketplace.

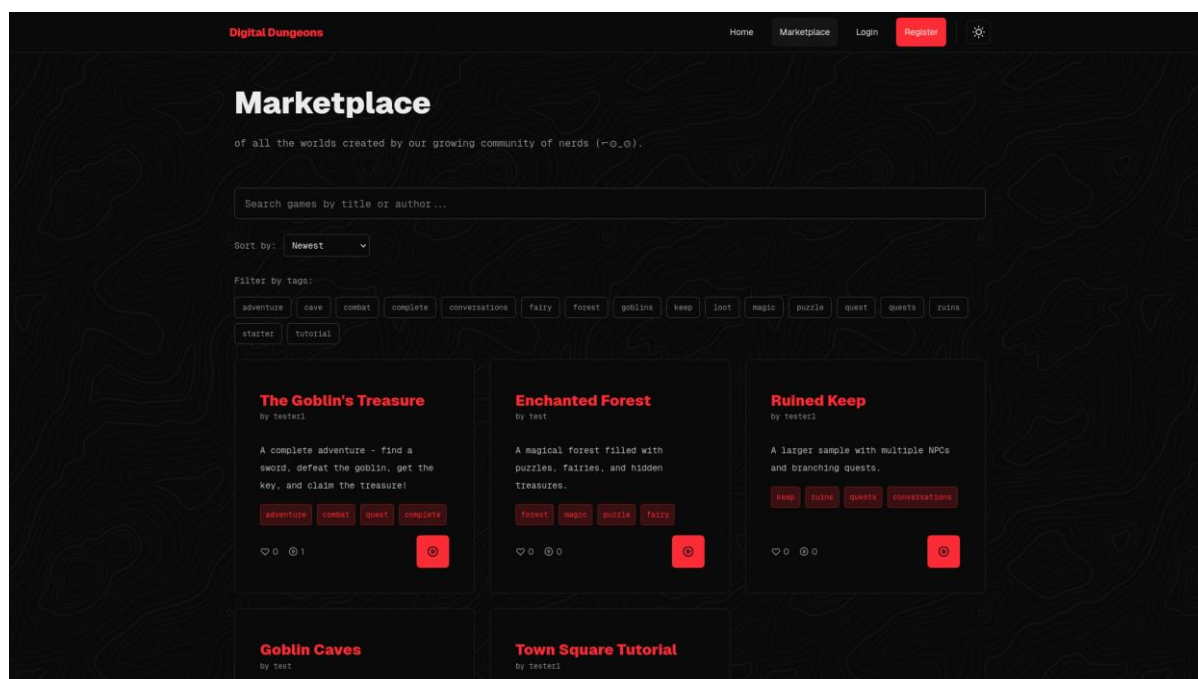


Rys. 7. Strona główna systemu Digital Dungeons



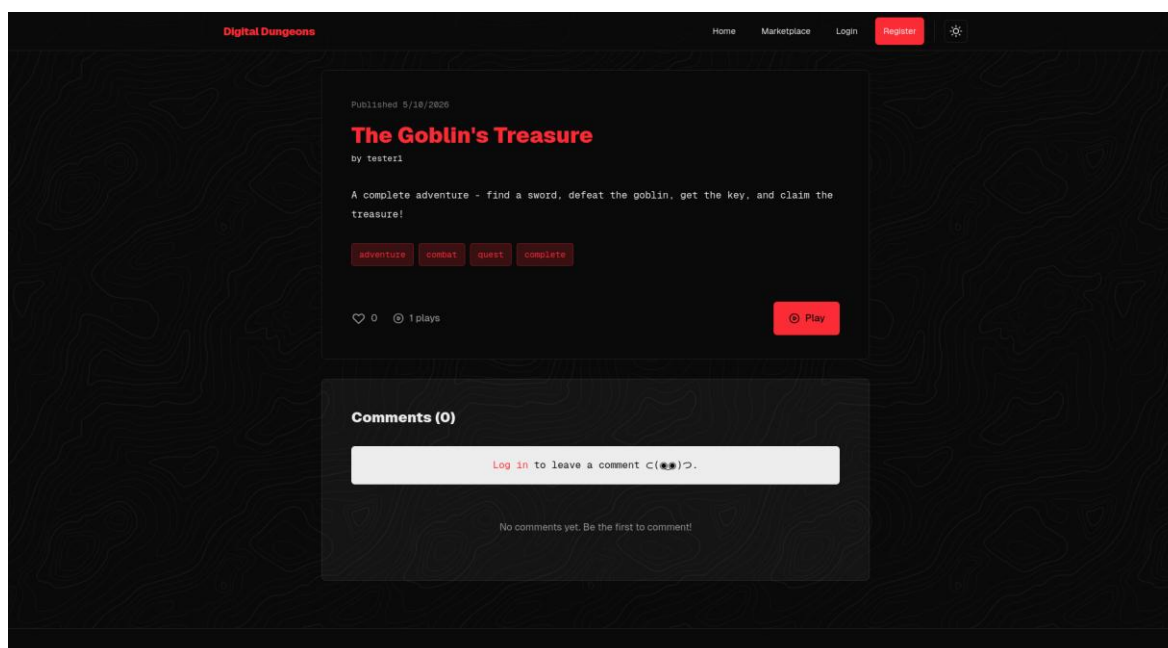
Rys. 8. Strona główna systemu Digital Dungeons w jasnej wersji

Ekran marketplace umożliwia przeglądanie opublikowanych gier, wyszukiwanie po tytule lub autorze, filtrowanie po tagach oraz sortowanie wyników według daty, tytułu, liczby rozgrywek i liczby polubień. Każda gra jest prezentowana w formie kafelka z najważniejszymi informacjami, a użytkownik może przejść do szczegółów konkretnego projektu.



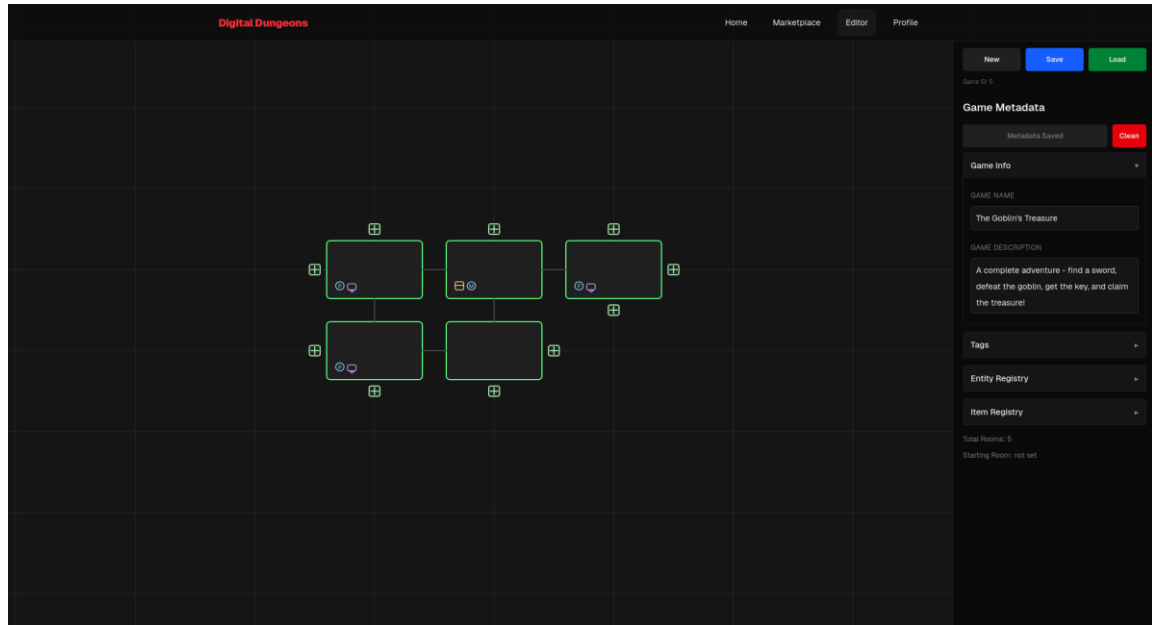
Rys. 9. Ekran marketplace

Widok szczegółów gry zawiera opis, metadane, tagi, licznik polubień i licznik uruchomień, a także sekcję komentarzy. Z poziomu tego ekranu każdy użytkownik (w tym niezalogowany gość) może uruchomić grę, natomiast polubienie, dodanie komentarza lub edycja własnego projektu wymagają zalogowania.



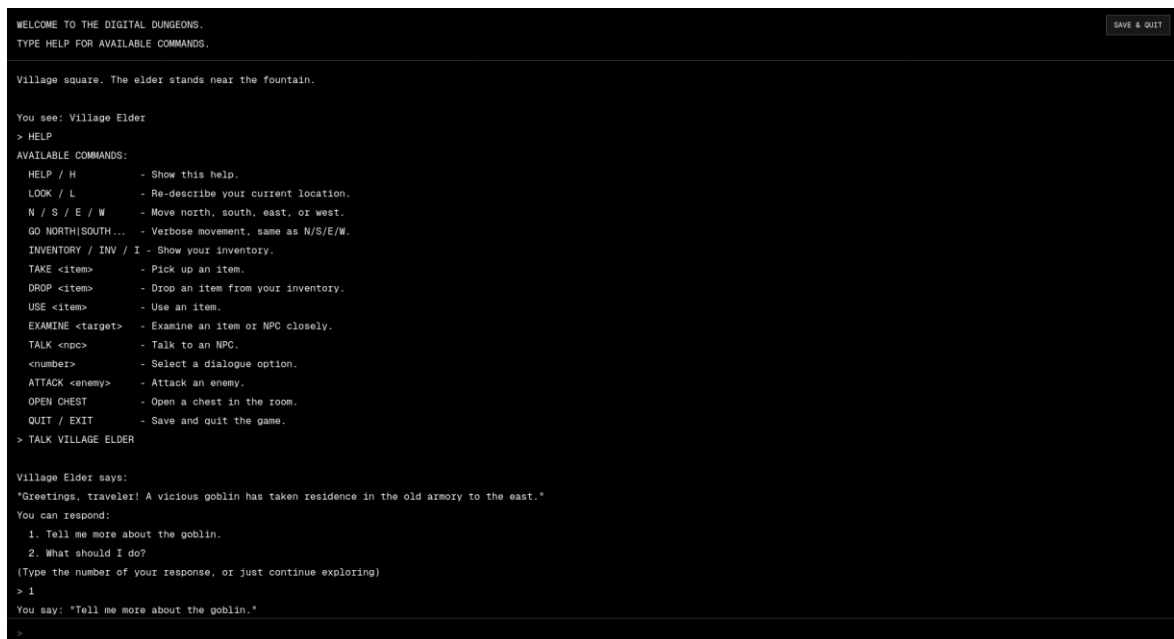
Rys. 10. Widok szczegółów gry

Edytor gier został zaprojektowany jako interfejs no-code. Składa się z płótna graficznego, panelu bocznego oraz osobnego trybu edycji konwersacji. Użytkownik może tworzyć pokoje, nadawać im nazwy i opisy, przypisywać obiekty i encje, definiować skrzynie oraz ustawienia startowe. Rozwiązanie to ma odciążyć twórcę od konieczności pisania kodu, pozwalając mu skoncentrować się na układzie świata oraz narracji.



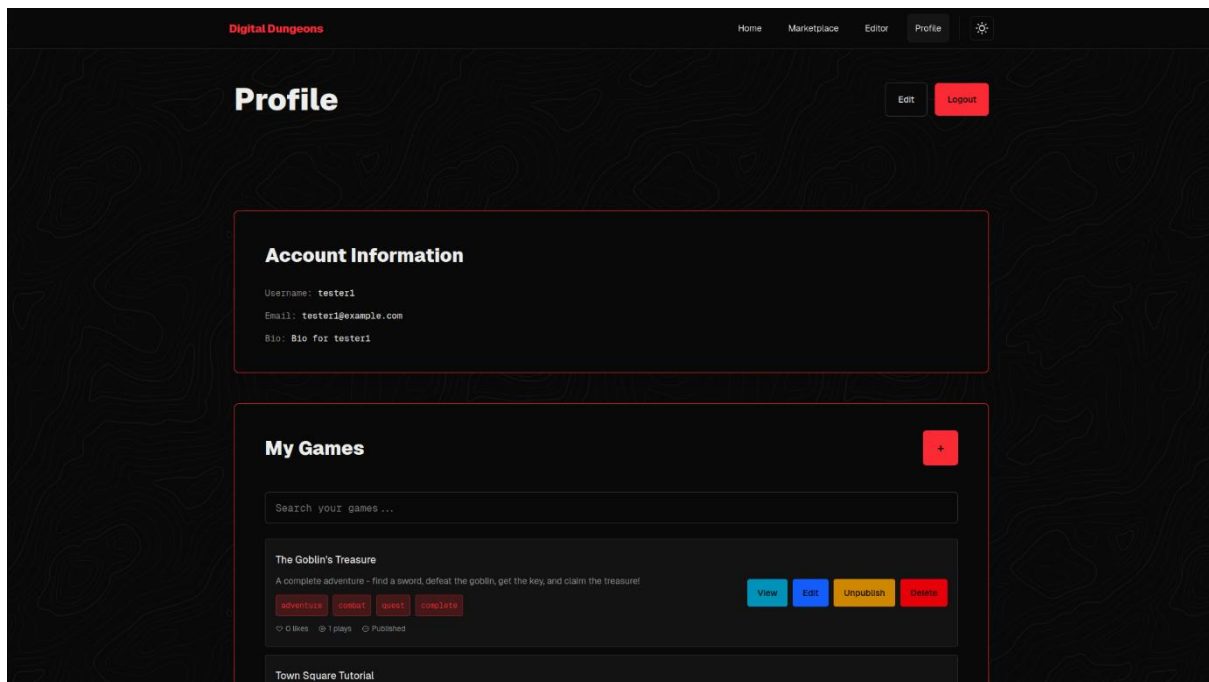
Rys. 11. Widok edytora gry

Widok rozgrywki przyjmuje formę klasycznej konsoli tekstowej. Minimalistyczna prezentacja sprzyja koncentracji na treści opisowej i komendach wpisywanych przez gracza. Taki projekt interfejsu jest spójny z charakterem gier tekstowych RPG oraz z założeniem ograniczenia elementów multimedialnych.



Rys. 12. Widok rozgrywki

Panel profilu użytkownika umożliwia zarządzanie danymi konta, przeglądanie własnych gier, obserwowanie polubionych tytułów oraz kontrolowanie publikacji. Dzięki temu użytkownik ma dostęp do wszystkich najważniejszych funkcji związanych z jego aktywnością w serwisie z jednego miejsca.



Rys. 13. Widok panelu profilu użytkownika

4.4. Projekt modułów systemu

System został podzielony na kilka współpracujących modułów, co ułatwia rozwój, testowanie oraz utrzymanie aplikacji. Każdy moduł odpowiada za wyodrębniony obszar funkcjonalny i komunikuje się z pozostałymi elementami wyłącznie przez zdefiniowane interfejsy.

4.4.1. Moduł użytkowników

Moduł użytkowników odpowiada za rejestrację, logowanie, zarządzanie profilem oraz pobieranie danych o bieżącym użytkowniku. W tym obszarze realizowane są także operacje związane z uwierzytelnianiem opartym na JWT oraz aktualizacją znacznika ostatniego logowania. Moduł ten stanowi podstawę dostępu do funkcji wymagających konta, takich jak publikowanie gier, komentowanie czy zapisywanie postępu rozgrywki.

4.4.2. Moduł marketplace gier

Moduł marketplace gier odpowiada za prezentację publicznie dostępnych projektów. Umożliwia pobieranie listy opublikowanych gier, filtrowanie ich po tagach, sortowanie, wyszukiwanie oraz obsługę polubień. W module tym realizowane są również operacje związane z podglądem szczegółów gry, jej komentarzy, licznika rozgrywek oraz najważniejsze, czyli rozpoczęcie rozgrywek.

Z punktu widzenia użytkownika marketplace pełni funkcję publicznego katalogu społeczności. Z punktu widzenia systemu jest to warstwa odczytu danych z tabel *games*, *likes* i *comments*, wsparta informacjami pobieranymi o autorach gier.

4.4.3. Moduł edytora gier

Moduł edytora gier obejmuje wizualny interfejs tworzenia świata gry. Pozwala on na definiowanie pokoi, ich opisów, przedmiotów, NPC, skrzyń, tagów oraz danych globalnych gry. W edytorze działa również osobny mechanizm tworzenia drzew konwersacji, powiązanych z wybranymi pokojami.

Moduł ten korzysta z komunikacji między *React* a *p5.js*, aby zachować zgodność między stanem kanwy, panelem bocznym i logiką zapisu. Po zapisaniu gry pełna definicja świata trafia do pola *game_content* w tabeli *games*. Dzięki temu edytor stanowi samodzielny obszar pracy twórcy, oddzielony od silnika rozgrywki.

4.4.4. Silnik rozgrywki

Silnik rozgrywki odpowiada za interpretację komend tekstowych gracza, aktualizację stanu świata oraz generowanie odpowiedzi tekstowych. Jego zadaniem jest odczyt definicji gry, przetworzenie bieżącej akcji użytkownika i zapis zmian do stanu sesji w tabeli *playthroughs*.

Silnik współpracuje z komponentem *GameConsole*, parserem komend, modulem *gameActions* (obsługa ekwipunku, walki i skrzyń) oraz modulem *roomHelpers*, w którym realizowana jest nawigacja między pomieszczeniami i generowanie opisów lokacji.

4.5. Projekt API systemu

API systemu zostało zaprojektowane jako zestaw endpointów REST udostępnianych pod prefiksem */api*. Warstwa ta obsługuje komunikację pomiędzy frontendem i backendem oraz stanowi jedyny kanał dostępu do danych aplikacji.

Moduł autoryzacji udostępnia operacje rejestracji, logowania oraz pobrania danych bieżącego użytkownika. Moduł gier obsługuje pobieranie listy opublikowanych gier, pobieranie pojedynczej gry, tworzenie, aktualizację, usuwanie oraz pobieranie gier konkretnego autora. Moduł użytkowników umożliwia pobieranie profilu, aktualizację bio, przeglądanie gier użytkownika, polubień, historii rozgrywek i statystyk. Moduł polubień pozwala dodać i usunąć like oraz sprawdzić, czy dany użytkownik polubił konkretną grę. Moduł komentarzy odpowiada za pobieranie komentarzy gry, dodawanie nowych wpisów, edycję i usuwanie własnych komentarzy. Moduł rozgrywki obsługuje uruchamianie, wznawianie, resetowanie, aktualizację oraz usuwanie sesji *playthrough*.

Ważnym elementem API jest mechanizm kontroli dostępu. Część operacji jest publiczna, jednak działania modyfikujące dane wymagają zalogowania. Token JWT jest automatycznie przesyłany przez przeglądarkę jako ciasteczko *httpOnly* i weryfikowany przez middleware *auth* po stronie serwera. Dodatkowo endpointy związane z edycją gier, komentarzy i sesji rozgrywki sprawdzają właściciela zasobu, dzięki czemu użytkownik może modyfikować wyłącznie własne dane.

Tak zaprojektowane API zapewnia spójność pomiędzy modułami frontendu i backendu oraz umożliwia obsługę wszystkich najważniejszych scenariuszy aplikacji w sposób uporządkowany i przewidywalny.

5. Implementacja systemu

W rozdziale opisano szczegóły implementacji systemu *Digital Dungeons*. Przedstawiono zastosowane technologie oraz sposób realizacji poszczególnych modułów aplikacji: frontendu, backendu, bazy danych, edytora gier, silnika rozgrywki, marketplace oraz systemu użytkowników.

5.1. Wykorzystane technologie

5.1.1. Frontend

Warstwa klienta została zaimplementowana z użyciem frameworka *Next.js* 16 [26] opartego na bibliotece *React* 19 [27]. Do renderowania graficznego edytora gier zastosowano bibliotekę *p5.js* 2 [28]. Stylizację zapewniono za pomocą *Tailwind CSS* 4 [29].

5.1.2. Backend

Warstwa serwerowa zbudowana jest na frameworku *Express.js* 5 [30] uruchomionym w środowisku Node.js [31]. Spośród bibliotek pomocniczych najważniejsze to: *bcryptjs* (haszowanie haseł), *jsonwebtoken* (tokeny JWT), *cookie-parser* (obsługa ciasteczek httpOnly), *express-validator* (walidacja danych wejściowych), *express-rate-limit* (ograniczanie żądań) oraz *helmet* (bezpieczne nagłówki HTTP). Komunikacja z bazą danych odbywa się przez bibliotekę *mysql2*.

5.1.3. Baza danych

Projekt wykorzystuje *MariaDB* [32] – relacyjną bazę danych zgodną z *MySQL*. Schemat obejmuje pięć tabel, których struktura i przeznaczenie zostały szczegółowo opisane w rozdziałach 3.7 oraz 4.2. Jest on inicjalizowany automatycznie przez środowisko Docker przy pierwszym uruchomieniu.

5.2. Implementacja interfejsu użytkownika

Interfejs użytkownika jest aplikacją zbudowaną w *Next.js* z użyciem *App Router*. Zaimplementowane widoki to: strona główna, marketplace, strona szczegółów gry, edytor, rozgrywka, profil oraz formularze logowania i rejestracji. Globalny stan uwierzytelnienia zarządzany jest przez *AuthContext* oparty na *React Context API*. Przy pierwszym renderowaniu aplikacja wysłała żądanie do */api/auth/me*. Przeglądarka automatycznie dołącza ciasteczko httpOnly, dzięki czemu sesja jest odtwarzana bez konieczności jawnego zarządzania tokenem w kodzie aplikacji klienckiej.

5.3. Implementacja edytora gier

Edytor składa się z płótna *p5.js* oraz panelu bocznego *React*. Płótno (*sketchFactory.js*) renderuje pokoje jako prostokąty na siatce współrzędnych (gx, gy) z obsługą przesuwania widoku i zoomu. Sąsiadujące pokoje są automatycznie połączone przejściami kierunkowymi. Komunikacja między *p5.js* a *React* odbywa się przez globalny obiekt *RPGEditorBridge* i niestandardowe zdarzenia DOM (*editor-state-snapshot*, *editor-selection-change*), co zachowuje niezależność obu warstw. Panel boczny (*EditorSidebar*) obsługuje edycję metadanych gry i pokoju, definiowanie encji, przedmiotów i skrzyń oraz zapis do API. Drzewa dialogowe NPC tworzone są w osobnym edytorze konwersacji (*ConversationCanvas.jsx*).

5.4. Implementacja silnika rozgrywki

Silnik rozgrywki działa w całości po stronie klienta, w komponencie *GameConsole* i modułach w *src/lib/game/*. Serwer jest odpytany wyłącznie przy wczytaniu gry i przy zapisie stanu. Po wpisaniu komendy funkcja *parseCommand()* normalizuje wejście (wielkie litery, podział na czasownik i argumenty), a następnie *GameConsole* przekazuje je do odpowiedniego handlera (*gameActions.js*, *roomHelpers.js*, *conversationSystem.js*). Po przetworzeniu stan gry jest aktualizowany lokalnie w *React*. Zapis do serwera przez *playthroughsApi.update()* następuje przy wyjściu z gry (komenda QUIT/EXIT lub przycisk "SAVE & QUIT").

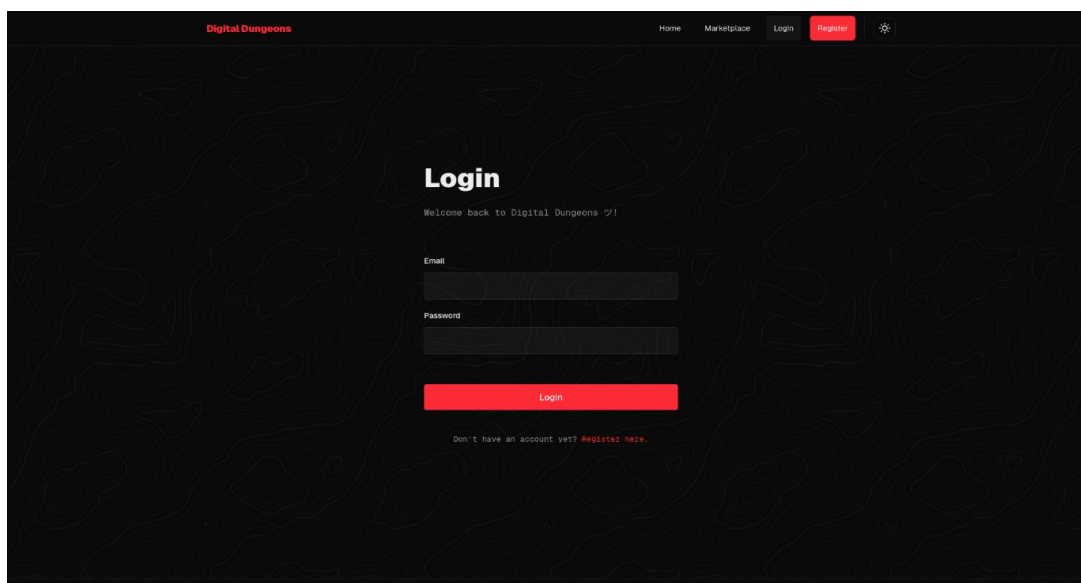
Obsługiwane komendy to m.in.: N / S / E / W oraz GO NORTH | SOUTH | EAST | WEST (ruch), LOOK, TAKE (aliasy: GET, PICKUP), DROP, USE, INVENTORY (aliasy: INV, I), EXAMINE (aliasy: INSPECT, X), ATTACK (aliasy: KILL, FIGHT), TALK (alias: SPEAK), OPEN CHEST, HELP oraz QUIT/EXIT (zapis i wyjście z gry).

5.5. Implementacja marketplace

Marketplace pobiera metadane wszystkich opublikowanych gier z */api/games* (z opcjonalnym filtrowaniem po tagach realizowanym po stronie serwera) i przetwarza je po stronie klienta. Dostępne opcje to: filtrowanie po tagach, wyszukiwanie po tytule lub autorze, sortowanie (najnowsze, najstarsze, tytuł A-Z, tytuł Z-A, polubienia, uruchomienia) oraz stopniowe wyświetlanie wyników po 12 gier przyciskiem „Load More” — bez dodatkowych żądań sieciowych.

5.6. Implementacja systemu użytkowników

Rejestracja waliduje nazwę użytkownika (od 3 do 50 znaków), format e-mail i hasło (min. 8 znaków), po czym haszuje hasło algorytmem *bcrypt* i ustawia ciasteczko *httpOnly authToken* z tokenem JWT ważnym 7 dni. Logowanie przyjmuje e-mail i hasło; oba endpointy chronione są przez rate limiter. Wylogowanie usuwa ciasteczko po stronie serwera. Profil umożliwia edycję bio, zarządzanie własnymi grami i przeglądanie polubionych tytułów.



Rys. 14. Formularz logowania

6. Testowanie systemu

W rozdziale przedstawiono przeprowadzone testy systemu *Digital Dungeons*. Opisano strategię testowania oraz wyniki weryfikacji warstwy frontendowej, endpointów API i kompletnych scenariuszy funkcjonalnych.

6.1. Strategia testowania

Testowanie oparto na manualnej weryfikacji scenariuszy przypadków użycia zdefiniowanych w rozdziale 3.5. Zrezygnowano z automatycznych testów jednostkowych ze względu na ograniczony czas realizacji projektu – kluczowe właściwości systemu najlepiej weryfikuje się w środowisku zbliżonym do produkcyjnego. Testy objęły warstwę frontendową, API backendowe oraz kompletne scenariusze end-to-end.

6.2. Testy frontendowe

Testy przeprowadzono manualnie w przeglądarkach Chrome i Firefox. Zweryfikowano poprawne renderowanie wszystkich widoków, działanie walidacji formularzy (błędne dane nie powodowały wysłania żądania do serwera), poprawne działanie edytora (dodawanie, edycja i usuwanie pomieszczeń, zapis i wczytanie gry), przełączanie motywu kolorystycznego oraz responsywność interfejsu.

6.3. Testy backendowe

Endpointy API weryfikowano za pomocą manualnych żądań HTTP. Żądania z niepoprawnymi danymi zwracały status 400, żądania bez tokenu zwracały status 401, próby modyfikacji cudzych zasobów zwracały status 403, a przekroczenie limitu prób logowania zwracało status 429. Potwierdzono obecność nagłówków bezpieczeństwa ustawianych przez *Helmet*.

6.4. Testy funkcjonalne systemu

Przeprowadzono testy end-to-end dla wszystkich scenariuszy przypadków użycia. Rejestracja i logowanie działały zgodnie z oczekiwaniami, w tym odrzucanie zduplikowanych danych i błędnych danych logowania. Tworzenie gry, zapis do bazy i ponowne wczytanie przebiegły poprawnie. W rozgrywce zweryfikowano ruch między pokojami, zarządzanie ekwipunkiem (TAKE / DROP / USE), walkę z encjami, otwieranie skrzyń z kluczem i bez, system dialogów NPC oraz zapis i wznowienie postępu. Publikacja gry czyniła ją widoczną w marketplace; jej cofnięcie usuwało ją z wykazu. Komentarze i polubienia działały poprawnie, w tym blokada pustych komentarzy i wielokrotnych polubień.

6.5. Wyniki testów

Wszystkie poddane ewaluacji scenariusze, bazujące bezpośrednio na opisach przypadków użycia wyszczególnionych w rozdziale 3.5, zwróciły wyniki zgodne z oczekiwanym zachowaniem systemu, zarówno dla ścieżek nominalnych, jak i przypadków brzegowych. Zgodnie z przyjętą strategią testowania (zob. rozdział 6.1), weryfikację przeprowadzono w całości manualnie, świadomie rezygnując z implementacji automatycznych testów jednostkowych na rzecz rygorystycznej walidacji w środowisku zbliżonym do produkcyjnego. Ponadto, testy bezpieczeństwa potwierdziły poprawne blokowanie nieautoryzowanego dostępu, skuteczność mechanizmu rate limitingu oraz właściwą konfigurację nagłówków HTTP. Wydajność operacyjna systemu została utrzymana na zadowalającym poziomie, a czasy odpowiedzi API bezbłędnie mieściły się w rygorze zdefiniowanym w wymaganiu WNF-04.

7. Podsumowanie i kierunki dalszego rozwoju

W niniejszym rozdziale przedstawiono podsumowanie pracy nad projektem oraz ocenę opracowanego systemu *Digital Dungeons*. Wskazano również ograniczenia wynikające z przyjętego zakresu pracy. W końcowej części zaprezentowano możliwe kierunki dalszego rozwoju platformy.

7.1. Podsumowanie realizacji projektu

Celem pracy było opracowanie systemu umożliwiającego tworzenie i publikowanie tekstowych gier typu RPG przez osoby bez doświadczenia programistycznego. W ramach realizacji projektu zaprojektowano oraz zaimplementowano platformę *Digital Dungeons*, dostępną w całości z poziomu przeglądarki internetowej.

System ten składa się z kilku głównych komponentów:

- Edytor gier: umożliwia tworzenie interaktywnych symulacji (gier) z narracją i zdefiniowanym przebiegiem rozgrywki.
- Silnik rozgrywki: odpowiada za interpretację działań gracza oraz zarządzanie stanem gry.

- Baza danych i moduł użytkowników: zarządza autoryzacją i przechowywaniem danych.
- Moduł publikacji i dystrybucji gier (marketplace): pozwala na udostępnianie utworzonych projektów społeczności.

Na podstawie przeprowadzonej implementacji oraz testów można stwierdzić, że założenia projektowe zostały w znacznym stopniu zrealizowane. System umożliwia tworzenie gier w modelu no-code, działa bezproblemowo w środowisku przeglądarkowym oraz zapewnia podstawowe mechanizmy niezbędne do prowadzenia rozgrywki, takie jak obsługa decyzji użytkownika oraz zapis stanu gry.

7.2. Ograniczenia systemu

Pomimo osiągnięcia założonych celów, opracowany system posiada pewne ograniczenia wynikające zarówno z przyjętego zakresu pracy, jak i podjętych decyzji projektowych.

Brak multimediów: Jednym z głównych ograniczeń jest brak wsparcia dla elementów multimedialnych, takich jak grafika, dźwięk czy wideo. System obejmuje jedynie model interakcji na bazie wprowadzanego za pomocą klawiatury tekstu, co znacząco utrudnia integrację tych elementów. Jest to jednak zgodne z pierwotnym założeniem o położeniu głównego nacisku na narrację tekstową.

Zależność od połączenia sieciowego: Kolejnym ograniczeniem jest dostępność systemu wyłącznie w środowisku przeglądarkowym. Brak możliwości eksportu gier w formie niezależnych aplikacji (ang. standalone) powoduje konieczność stałego połączenia z siecią podczas korzystania z platformy.

Uproszczony edytor: Ograniczenia dotyczą również funkcjonalności edytora, który oferuje stosunkowo prosty model tworzenia gier. Choć zwiększa to dostępność *Digital Dungeons* dla początkujących użytkowników, jednocześnie ogranicza możliwości konstruowania wysoce złożonych scenariuszy i mechanik.

Brak wsparcia AI: Dodatkowo system obecnie nie wykorzystuje mechanizmów automatycznego generowania treści, takich jak rozwiązania oparte na sztucznej inteligencji, które mogłyby wspierać twórców w pisaniu opisów lokacji czy dialogów.

7.3. Możliwości dalszego rozwoju

Opracowany system stanowi solidną podstawę, którą można rozwijać zarówno pod kątem funkcjonalnym, jak i technicznym.

Integracja multimediów: Jednym z możliwych kierunków rozwoju jest rozszerzenie systemu o opcjonalne wsparcie dla elementów multimedialnych, takich jak prosta oprawa graficzna czy podkłady dźwiękowe. Umożliwiłoby to zwiększenie różnorodności tworzonych gier oraz poszerzenie grupy docelowych odbiorców.

Rozbudowa mechanik narracyjnych: Inną możliwością jest rozbudowa mechanizmu narracji i logiki, co pozwoliłoby na tworzenie bardziej skomplikowanych fabuł. Taka rozbudowa mogłaby obejmować wprowadzenie zaawansowanych systemów zarządzania ekwipunkiem, statystykami postaci czy zmiennymi warunkowymi w dialogach.

Eksport do zewnętrznych formatów: Istotnym kierunkiem rozwoju jest możliwość eksportu gotowych gier do postaci plików wykonywalnych (niezależnych aplikacji) lub standardowych formatów webowych do osadzenia na innych stronach. Pozwoliłoby to na uniezależnienie rozgrywki od środowiska platformy.

Skalowalność i wydajność: W zakresie technicznym wskazane jest wprowadzenie optymalizacji wydajnościowych oraz rozwiązań chmurowych zwiększających skalowalność systemu. Byłoby to kluczowe w przypadku znacznego wzrostu liczby aktywnych użytkowników i publikowanych w serwisie gier.

8. Bibliografia

- [1] Raport *Video Games Europe 2023 Key Facts Report*, https://www.videogameseurope.eu/wp-content/uploads/2024/09/Video-Games-Europe-2023-Key-Facts-Report_FINAL.pdf, 09.2024.
- [2] Statystyki publikacji gier na platformie *Steam*, <https://steamdb.info/stats/releases/>, 04.2026.
- [3] Program *TADS*, <https://www.tads.org/>, 04.2026.
- [4] Program *Adrift*, <https://www.adrift.co/>, 04.2026.
- [5] Silnik *Unity*, <https://unity.com/>, 04.2026.
- [6] Silnik *Unreal Engine*, <https://www.unrealengine.com/>, 04.2026.
- [7] Waga silnika *Unreal Engine*, <https://forums.unrealengine.com/t/unreal-engine-5-02-size-115-gb/593895>, 04.2026.
- [8] Program *RPG Maker*, <https://www.rpgmakerweb.com/>, 04.2026.
- [9] Program *Inform 7*, <https://ganelson.github.io/inform-website/downloads/>, 04.2026.
- [10] Program *Twine*, <https://twinery.org/>, 04.2026.
- [11] Platforma *Steam*, <https://store.steampowered.com/>, 04.2026.
- [12] Gra *Cyberpunk 2077*, https://store.steampowered.com/app/1091500/Cyberpunk_2077/, 04.2026.
- [13] Cennik publikacji gry na platformie *Steam*, <https://steamcommunity.com/groups/steamworks/announcements/detail/1697191267930157838>, 04.2026.
- [14] Sklep *Epic Games Store*, <https://store.epicgames.com/>, 05.2026.
- [15] Podział przychodów w *Epic Games Store*, <https://store.epicgames.com/pl/news/epic-games-store-launches-self-publishing-tools-for-game-developers-and-publishers>, 05.2026.
- [16] Narzędzia do publikacji *Epic Games*, <https://dev.epicgames.com/pl/distribute>, 05.2026.
- [17] Platforma *GOG.com*, <https://www.gog.com/>, 05.2026.
- [18] Platforma *itch.io*, <https://itch.io/>, 04.2026.
- [19] Platforma *Game Jolt*, <https://gamejolt.com/>, 05.2026.
- [20] Przewodnik publikacji *HTML5* na *itch.io*, <https://itch.io/docs/creators/html5>, 05.2026.
- [21] Konsola *Google Play*, <https://play.google.com/console/about/>, 05.2026.
- [22] *Apple Developer Program*, <https://developer.apple.com/programs/>, 05.2026.
- [23] Wytyczne publikacji *App Store*, <https://developer.apple.com/app-store/review/guidelines/>, 05.2026.
- [24] Strona *Choice of Games*, <https://www.choiceofgames.com/>, 05.2026.
- [25] Platforma *TextAdventures.co.uk*, <https://textadventures.co.uk/>, 05.2026.

- [26] Dokumentacja *Next.js*, <https://nextjs.org/docs>, 05.2026.
- [27] Dokumentacja *React*, <https://react.dev>, 05.2026.
- [28] Dokumentacja *p5.js*, <https://p5js.org>, 05.2026.
- [29] Dokumentacja *Tailwind CSS*, <https://tailwindcss.com/docs>, 05.2026.
- [30] Dokumentacja *Express.js*, <https://expressjs.com>, 05.2026.
- [31] Dokumentacja *Node.js*, <https://nodejs.org>, 05.2026.
- [32] Dokumentacja *MariaDB*, <https://mariadb.org/documentation>, 05.2026.